Project repository: https://github.com/Sallar-Khan/Cybersecurity-Cohort-02---PSEB
This module: https://github.com/Sallar-Khan/Cybersecurity-Cohort-02---PSEB/tree/main/s09-web-recon

PSEB — Cyber Security

Web Application Reconnaissance and Scanning

Assignment report: scope document, dork report, subdomain reconciliation and scanner comparison

Item	Detail
Student	Sallar Ali Ahmad Khan
Course	PSEB — Cyber Security
Submitted	11 November 2026

Contents

Task	Requirement	Weight
Task 1 — Scope document	Scope and rules of engagement for a hypothetical *.intel.com engagement, including exclusions, the break-glass clause and the emergency procedure	25%
Task 2 — Dork report	Six exposures on an authorised domain: each dork, its result and the fix	25%
Task 3 — Subdomain reconciliation	Three tools, one deduplicated list, what each tool found alone and why	25%
Task 4 — Scanner comparison	ZAP and Burp against the same target: what each found that the other missed, and why	25%

Tasks 2 and 3 were carried out on a private bug bounty program to which the student holds an invitation. In line with that program's non-disclosure rule, the company, its domain and third-party names are replaced with placeholders such as target.example. Task 1 is hypothetical; Task 4 used a local, deliberately vulnerable lab.

Task 1 — Scope and Rules of Engagement: *.intel.com Web Assessment

Hypothetical engagement prepared for a training assignment. All names, phone numbers, email addresses and IP addresses are fictional. Intel has not authorized any testing.

1. Document control and authorization

Intel Corporation authorizes Devsecplus to perform an external web application security assessment of *.intel.com from 1 to 11 December 2026, under the rules in this document.

Item	Detail
Client	Intel Corporation ("the Client")
Tester	Devsecplus ("the Tester")
Engagement ID	DSP-INTEL-2026-001
Engagement type	External, authenticated and unauthenticated web application test (grey box)
Testing period	1 December 2026 to 11 December 2026
Client authorizing officer	John Doe, Director of Product Security
Tester lead	Sallar Ali Ahmad Khan, Lead Penetration Tester
Document version	1.0

Testing may only begin after both parties sign Section 8. If any part of this document conflicts with a verbal instruction, this document applies until a signed amendment replaces it.

The Client confirms that it owns, or has the legal right to authorize testing of, every asset listed as in scope. The Tester's permission covers only those assets, those dates and the activities in Section 5.

2. Objective and methodology

The goal is to find and report exploitable weaknesses in Intel's internet-facing web applications before a real attacker does, without disrupting the business.

The Tester will follow the OWASP Web Security Testing Guide (WSTG) v4.2 and the PTES phases:

1. **Reconnaissance**: passive discovery of subdomains, technologies and exposed content.
2. **Mapping**: crawling in-scope applications and identifying entry points.
3. **Vulnerability discovery**: automated scanning (OWASP ZAP, Burp Suite Professional) plus manual testing.
4. **Controlled exploitation**: proving a vulnerability exists with the least intrusive method possible.
5. **Reporting**: findings, evidence, risk ratings (CVSS v3.1) and fixes.

The Client will provide two test accounts per application (one standard user, one administrator) so authenticated areas can be tested without using real customer accounts.

3. In scope

Only web services under *.intel.com that resolve to IP addresses owned or controlled by Intel are in scope. Before testing each host, the Tester must check that its IP address appears on the Client's asset list (Appendix A, supplied by the Client) and that its DNS does not point to a third-party provider.

What the wildcard covers

- All subdomain levels, for example `shop.intel.com` and `api.dev.intel.com`.
- TCP ports 80 and 443 only. Any other open port is reported but not tested.

- The apex domain `intel.com` itself.

Testing conditions

Condition	Rule
Testing window	Monday to Friday, 09:00 to 18:00 PKT (UTC+5)
Blackout dates	None in this period; the Client may add some with 24 hours' notice
Source IP addresses	203.0.113.10, 203.0.113.11, 203.0.113.12 (Tester will not test from any other address)
Rate limit	Maximum 10 requests per second per host; maximum 2 scanners running at once
Identifying header	Every request includes the header `X-Pentest: DSP-INTEL-2026-001`
Test accounts	Only accounts supplied by the Client; never real customer or employee accounts

The Client's Security Operations Centre (SOC) will be told the source IP addresses and the header value before testing starts, so it can tell this test apart from a real attack.

4. Exclusions (out of scope)

Anything not listed in Section 3 is out of scope, even if it looks like part of Intel. The exclusions below apply even where a host matches *.intel.com. Hostnames are illustrative examples.

Excluded assets

Excluded item	Reason
Subdomains that point (CNAME) to third-party services, e.g. a careers site on a hiring platform, a support portal on a SaaS helpdesk, marketing pages on a CDN or email platform	The servers belong to another company. Intel cannot authorize testing of systems it does not own.
Production payment and order systems, e.g. checkout or billing hosts	A failed test could block real customer payments and create PCI DSS compliance issues.
Single sign-on and corporate identity providers	Lockouts would stop thousands of employees from working.
Manufacturing, factory and operational technology (OT) systems	Disruption could stop physical production lines and create safety risks.
Internal networks reached through a vulnerability (no pivoting)	The engagement is external only; internal testing needs a separate agreement.
Employee personal devices, accounts and social media	Not company property; testing them would breach privacy laws.
Domains other than intel.com (e.g. other top-level domains or acquired brands)	Not named in this agreement.
Any asset added to the estate after 1 December 2026	Ownership and risk have not been reviewed.

Excluded activities

Excluded activity	Reason
Denial-of-service, stress or load testing	Could take services offline for real users.
Social engineering: phishing, phone calls, impersonation	Targets people, not systems, and needs separate HR and legal approval.

Excluded activity	Reason
Physical access to Intel buildings or data centres	Outside the web assessment.
Brute-forcing real user passwords or credential stuffing	Would lock out real accounts and use leaked third-party data.
Changing, deleting or encrypting data	Could cause permanent loss; proof of access is enough.
Installing backdoors, web shells or persistence	Leaves the Client less secure than before the test.
Spam or mass email through forms	Could get Intel's mail servers blocklisted.

5. Permitted activities and data handling

The Tester may use any standard web testing technique on in-scope assets, provided it stays inside the limits in Sections 3 and 4.

Permitted

- Passive reconnaissance: DNS records, certificate transparency logs, search engine queries, public code repositories.
- Crawling and automated vulnerability scanning within the rate limit.
- Manual testing for the OWASP Top 10, including injection, cross-site scripting, broken access control and server-side request forgery.
- Proof-of-concept exploitation that reads the minimum data needed to prove the issue, for example a database version string or one row count.

Data handling rules

1. **Least data.** If personal, financial or credential data is reachable, prove access with a screenshot showing the field names or a record count, not the values. Then follow Section 6.
2. **Storage.** All evidence is stored only on Devsecplus laptops with full-disk encryption (AES-256), and never on personal devices or public cloud storage.
3. **Transfer.** Reports and evidence are shared with the Client only through the encrypted channel agreed in Section 8.
4. **Retention.** All Client data and evidence is securely deleted 30 days after the final report is accepted, and the Tester confirms deletion in writing.
5. **Confidentiality.** Findings are not disclosed to anyone outside the named contacts, and not published, without the Client's written consent.

6. Break-glass clause

If any trigger below happens, the Tester must stop all testing at once, across every host, and start the emergency procedure in Section 7. No permission is needed to stop; permission is needed to restart.

Stop triggers

Trigger	Example	Why testing must stop
Evidence of a real attacker	An unknown web shell, unfamiliar admin accounts, attacker tools on a server	The Client faces an active incident, and further testing could destroy forensic evidence.
Sensitive data exposed	Customer personal data, passwords, payment card numbers, health or export-controlled data	The Client may have legal reporting duties, and the Tester must not see more.
Service impact	A site slows down, returns errors or goes offline during testing	The test may be causing harm to real users.

Trigger	Example	Why testing must stop
Critical public risk	Unauthenticated remote code execution, or a database open to the internet	Real attackers could exploit it at any moment; the Client must fix it now, not at report time.
Scope breach	The Tester realises a host is third-party owned or otherwise excluded	Continuing would be unauthorized access.
Safety or legal concern	Any risk to physical safety, or a request from law enforcement	Outside what a web test should handle.

Who can stop the test

- The Tester lead or any Devsecplus team member, at any time.
- The Client's authorizing officer, primary or backup contact, or the SOC duty manager, at any time and without giving a reason.

A stop order from the Client can be given by phone and is effective immediately; it is confirmed by email afterwards.

7. Emergency procedure

The Tester must reach a Client contact by phone within 15 minutes of a break-glass trigger, and confirm in writing within 1 hour.

Steps

1. **Stop (0 minutes).** Stop every scanner and manual test immediately. Do not close sessions or delete anything on the target.
2. **Record (within 5 minutes).** Note the time (UTC), the host, the exact request that caused or found the issue, and what was seen. Save tool logs and screenshots.
3. **Call (within 15 minutes).** Phone the primary contact. If there is no answer within 15 minutes, call the backup contact, then the SOC hotline. Keep calling down the list until a person confirms they have the details.
4. **Do not investigate or fix.** Do not explore further, run more commands or try to repair anything, even if asked informally. Wait for written instructions.
5. **Confirm in writing (within 1 hour).** Email the details from step 2 to the primary contact and the SOC, using the encrypted channel.
6. **Support the Client.** Hand over logs and source IP addresses if the Client opens an incident investigation.
7. **Resume only with written approval** from the Client's authorizing officer. Record the approval and time in the engagement log.

Escalation contacts

Order	Role	Name	Phone	Email	Available
1	Client primary contact (Product Security Manager)	Daniel Reeves	+1-000-000-0001	primary@client.example	Testing hours
2	Client backup contact (Security Engineering Lead)	Robert Hale	+1-000-000-0002	backup@client.example	Testing hours
3	Client SOC hotline (duty manager)	Julian Max	+1-000-000-0003	soc@client.example	24/7

Order	Role	Name	Phone	Email	Available
4	Client authorizing officer (Director of Product Security)	John Doe	+1-000-000-0004	director@client.example	Escalation only
—	Tester lead, Devsecplus	Sallar Khan	+92-000-0000000	lead@devsecplus.example	24/7 during engagement
—	Devsecplus account manager	TA Khan	+92-000-0000001	manager@devsecplus.example	Business hours

8. Communication, reporting and sign-off

The Tester sends a short status update every testing day and a full report within 5 working days of the test ending.

Item	Detail
Daily status	Email by 18:30 PKT: hosts tested, tools used, any critical or high findings
Critical findings	Reported by phone the same day, never held for the final report (see Section 6)
Secure channel	PGP-encrypted email, or a shared folder the Client controls with multi-factor authentication
Final report	Due 18 December 2026: executive summary, each finding with evidence, CVSS v3.1 score and fix
Retest	One free retest of fixed findings within 60 days of the report
Changes to scope	Only by a written amendment signed by both parties

Sign-off

By signing, both parties agree to the scope and rules in this document.

	Client: Intel Corporation	Tester: Devsecplus
Name	John Doe	Sallar Ali Ahmad Khan
Title	Director of Product Security	Lead Penetration Tester
Signature	JDoe	SallarAAKhan
Date	10 November 2026	10 November 2026

Task 2 — Dork Report (Private Bug Bounty Target)

1. Summary

Ten passive searches across Google, the Wayback Machine and GitHub found six exposures on the target's sandbox environment, and confirmed three controls that are working. The most significant finding is API keys appearing in public archives and public code (Findings 3 and 5).

Authorization. The target is a private program on a bug bounty platform. The tester holds a personal invitation to it. Only passive techniques were used: every search went to a search engine, archive or code host, never to the target itself. No credential found was opened, tested or used.

Redaction. The program forbids disclosing vulnerability information without written consent. So in this report:

- The company is called **the Target** and its domain **target.example**.
- The sandbox environment is called **sandbox**, so hosts appear as `app.sandbox.target.example`, `api.sandbox.target.example` and so on.
- Third-party organisations and repository names are anonymised.
- All keys show only their first 4 characters.
- Screenshots of each result were taken with the domain, logos, keys and browser details blacked out. They are held by the tester and are not embedded here, so that no identifying detail is published.

Item	Detail
Target	Private bug bounty program, sandbox environment (5 in-scope hosts)
Testing date	10 November 2026, 20:15–20:40 PKT
Sources	Google Search, Internet Archive (Wayback CDX API), GitHub code search
Searches run	10 (5 returned results, 3 returned nothing, 2 failed because of an Internet Archive outage)
Findings	6 (2 Low–Medium, 4 Informational/Low)

2. Method and dork log

Google held only 3 URLs for the sandbox, so recon moved to public archives and public code once Google was exhausted.

1. **Google** with `site:` and file, title and URL operators. Searches were repeated with `&filter=0` (show omitted results) and `&pws=0` (no personalisation), which revealed two extra URLs.
2. **Wayback Machine CDX API**, which lists every archived URL for a host. The wildcard query timed out, so each host was queried separately.
3. **GitHub code search** for the sandbox hostname and for the domain with `apikey`.

Every search is logged below, including those that found nothing.

#	Source	Dork / query	Time (PKT)	Result	Finding
1	Google	`site:sandbox.target.example`	20:18	1 URL shown, others omitted	F1
2	Google	Same, with `&filter=0&pws=0`	20:21	3 URLs	F1, F2
3	Google	`site:sandbox.target.example inurl:graphql OR inurl:api`	20:23	0	Control working
4	Google	`site:sandbox.target.example ext:js OR ext:json OR ext:txt OR ext:xml OR ext:pdf`	20:25	0	Control working

#	Source	Dork / query	Time (PKT)	Result	Finding
5	Google	`site:sandbox.target.example intitle:"index of" OR intext:"error"`	20:25	0	Control working
6	Wayback	`cdx?url=*.sandbox.target.example/*&collapse=urlkey`	20:27	504 timeout	Retried per host
7	Wayback	`cdx?url=app.sandbox.target.example/*&limit=1000`	20:29	10 URLs	F3, F4
8	Wayback	Same for `api`, `keystone`, `verify`, `monitoring` hosts	20:30	Archive offline	Not completed
9	GitHub	`"sandbox.target.example"` (code)	20:34	34 code results	F5, F6
10	GitHub	`"target.example" apikey` (code)	20:36	922 code results	F5

In the real searches, `sandbox.target.example` was the program's actual sandbox hostname.

3. Findings 1–3

Finding 1: Unlisted test host indexed by Google

Item	Detail
Dork	`site:sandbox.target.example` (log #1, #2)
Result	Google indexed `title.sandbox.target.example/X-Cache`. This host is **not** in the program's asset list. It redirects to the sandbox login page, which displays "This is a testing site".
Severity	Informational / Low

Risk. Attackers use search engines to find test environments because they are often less protected than production. The result also revealed a hostname missing from the official asset inventory, and an odd indexed path (`/X-Cache`, an HTTP header name) that suggests a misconfigured redirect or cache rule.

Fix.

- Send `X-Robots-Tag: noindex, nofollow` on every response from test environments.
- Restrict test hosts to an IP allowlist or require login before any page loads.
- Remove existing results with Google Search Console's Removals tool.
- Add the host to the asset inventory, or retire it if it is no longer needed.

Finding 2: SAML single sign-on endpoints indexed

Item	Detail
Dork	`site:sandbox.target.example` with `&filter=0&pws=0` (log #2)
Result	Google indexed `api.sandbox.target.example/saml/consume` (receives login assertions) and `/saml/metadata` (service provider configuration).
Severity	Informational / Low

Risk. This shows the application uses SAML for single sign-on and where its endpoints are. SAML metadata normally reveals the entity ID, signing certificates and assertion URLs. Attackers use this to plan SAML attacks such as signature wrapping, XML injection or assertion replay. Those lead to account takeover, which the program lists among its worst-case scenarios.

Fix. Metadata is often public by design, so the aim is to reduce exposure, not hide it.

- Send `X-Robots-Tag: noindex` on all `/saml/*` paths and remove indexed results.
- If only known identity providers need it, restrict `/saml/metadata` to their IP addresses.
- Make sure `/saml/consume` strictly validates signatures, rejects replayed assertions and uses a maintained SAML library.

Finding 3: API keys in archived URLs

Item	Detail
Dork	Wayback CDX for `app.sandbox.target.example/*` (log #7)
Result	Three archived URLs of the form `/easy-link?ApiKey=8QG7****`, `NJgk****`, `ZbPD****`, captured between December 2025 and April 2026.
Severity	Low–Medium (depends on what the key permits; not tested)

Risk. Keys placed in a URL end up in browser history, server and proxy logs, analytics tools and public archives, where anyone can find them. Finding 5 suggests "easy link" keys are meant to be embedded on customer websites, so they may be limited "publishable" keys. Their real impact could not be confirmed without using them, which the tester did not do.

Fix.

- Confirm what each easy-link key can do. If it can do more than start a public signing flow, revoke and rotate the three keys.
- Pass keys in an `Authorization` header or a POST body, not in the URL.
- Use short-lived or single-purpose tokens for links meant to be shared publicly.
- Ask the Internet Archive to exclude URLs containing `ApiKey=`.

4. Findings 4–6

Finding 4: Security policy reveals third-party services and weak script rules

Item	Detail
Dork	Wayback CDX for `app.sandbox.target.example/*` (log #7)
Result	One archived "URL" is the site's Content-Security-Policy (CSP) header, captured by mistake. It lists session recording (FullStory), identity verification (Persona), payments (Stripe, Plaid, Google Pay), product analytics (Pendo), video (Wistia), advertising (Google, DoubleClick) and an internal host `inquiry.target.example`. Its `script-src` includes `'unsafe-eval'` and `'unsafe-inline'`.
Severity	Low / Informational

Risk. An attacker learns the payment and identity providers, which helps with convincing phishing and supply-chain targeting. `'unsafe-inline'` and `'unsafe-eval'` weaken the CSP's protection against cross-site scripting (XSS), so any injection bug becomes easier to exploit.

Fix.

- Remove `'unsafe-inline'` and `'unsafe-eval'`. Use nonces or hashes together with `'strict-dynamic'`, which the policy already partly uses.
- Remove third-party domains that are no longer needed.

- Keep session recording and advertising scripts off pages that handle identity documents or payments.

Finding 5: Easy-link API keys hard-coded in a customer's public GitHub repository

Item	Detail
Dork	GitHub code search `"sandbox.target.example"` and `"target.example" apikey` (log #9, #10)
Result	A public repository belonging to a government customer of the Target contains HTML pages with `app.sandbox.target.example/easy-link?ApiKey=****` links (three keys: `VPHS****`, `m5hS****`, `3s7d****`). The links sit inside HTML comments, so they are hidden on the live page but fully visible in the source.
Severity	Low–Medium (same uncertainty as Finding 3)

Risk. Commenting out a link does not hide it: anyone can read the source on GitHub or in the browser. The exposure sits with a customer, so the Target cannot fix it alone. It also confirms the easy-link key pattern seen in Finding 3.

Fix.

- **Target:** join GitHub's secret-scanning partner program, so keys matching its format are reported automatically when pushed to public repositories. Give easy-link keys a recognisable prefix to make this possible.
- **Target:** document for customers that easy-link keys must not be committed, and offer a quick way to rotate them.
- **Customer:** delete the commented-out links and rotate the keys.

Finding 6: Public SDKs and tools reveal API endpoints and token format

Item	Detail
Dork	GitHub code search `"sandbox.target.example"` (log #9)
Result	Community SDKs and command-line tools (Go, .NET, TypeScript), API specs and a secret-scanning rule file disclose: the OAuth token endpoint `/oauth/v2/token`; mortgage and transaction paths such as `/mortgage/v2/transactions/verify_address` and `/v1/transactions`; that sandbox tokens contain `_test_`; and two further hostnames not in the program's scope list.
Severity	Informational

Risk. An attacker gets an API map without sending any traffic to the Target. Knowing the token format makes leaked tokens easier to recognise when searching other public sources. Much of this is normal for a public developer API, so the risk is low, but it shortens an attacker's reconnaissance.

Fix.

- Accept that public API documentation is intentional, but publish an official SDK so developers don't rely on unofficial copies that may be outdated.
- Make sure every endpoint revealed here requires authentication and enforces per-tenant access checks.
- Use a token prefix that secret scanners can detect (as for Finding 5), and alert when a sandbox token is used against production.

5. Controls working, limitations and ethics

The Target keeps its sandbox almost entirely out of Google; the real exposure comes from archives and third-party code that it doesn't control.

Controls confirmed working

Dork	What it shows
#3 `inurl:graphql OR inurl:api` returned 0	The GraphQL API and API documentation are not indexed.
#4 file extensions returned 0	No scripts, configuration or documents are indexed.
#5 `intitle:"index of" OR intext:"error"` returned 0	No directory listings or error pages are indexed.

Limitations

- Google held only 3 URLs for the sandbox, so narrower Google dorks could only return subsets of those.
- The Internet Archive went offline during testing, so four of the five in-scope hosts were not checked in Wayback (log #8).
- Key impact (Findings 3 and 5) was not confirmed, because that would mean using credentials, which is outside passive recon and outside the program's rules.

Ethics and handling

- Only passive sources were queried. The only page opened on the Target was the public login page reached from the Google result.
- No key, token or URL containing a key was opened, tested or used.
- Findings 3 and 5 are candidates to report to the program through the platform. Under its rules, keys "leaked to third parties" and missing security headers are out of scope for rewards, so Findings 4 and 6 are for this assignment only.
- This report is redacted to follow the program's non-disclosure rule. The unredacted log and screenshots are kept encrypted and deleted after grading.

Task 3 — Subdomain Reconciliation (Private Bug Bounty Target)

1. Summary

Three passive tools found **229 unique names** for the Target's domain. Subfinder found almost all of them (228), crt.sh found 225 real names plus 10 false positives, and Amass found 13.

Tool	Names found	Found only by this tool
Subfinder	228	4
crt.sh (Certificate Transparency)	225 real + 10 noise	0
Amass (passive)	13	0 (only the apex domain, which Subfinder never lists)
Combined, deduplicated	**229**	

Authorization. The target is a private bug bounty program the tester was invited to. All three tools are passive: they query third-party databases (certificate logs, search engines, DNS datasets), so no traffic was sent to the Target. No host was opened, scanned or probed.

Redaction. The program forbids disclosure without consent, so the domain is written as **target.example** and the in-scope sandbox environment as **sandbox.target.example**. Real names were used when running the tools.

2. Method

The three tools were chosen because each collects names in a different way. All were run on Kali Linux on 10 November 2026, between 20:45 and 21:24 PKT.

Tool	Version	How it finds names	Command
Subfinder	Kali Linux package (exact version not recorded)	Queries many passive sources at once: certificate logs, passive DNS, search engines, threat-intelligence feeds	`subfinder -d target.example -silent -o subfinder.txt`
Amass	v5.1.1	Passive mode: its own set of data sources, stored in a local graph database	`amass enum -passive -d target.example -timeout 15`, then `amass subs -names -d target.example > amass.txt`
crt.sh	Web API	Certificate Transparency logs only: every name ever written on a public TLS certificate	`curl -s "https://crt.sh/?q=%25.target.example&output=json" | jq -r '.[].name_value'`, lowercased, `*.` removed, sorted

Problems met and how they were handled

- Amass v5 no longer supports `-o`; results were exported from its database with `amass subs` instead.
- Amass stalled at 99.97% while waiting on slow sources, so it was stopped with Ctrl+C after about 25 minutes. Its database kept everything found up to then.
- Files copied from Kali had Windows line endings, which made identical names look different. They were stripped (`sed 's/\r$//'`) before comparing.

Reconciliation steps

1. Normalise every file: lowercase, remove `*.` wildcard prefixes, remove line endings, sort and deduplicate.

2. Remove noise that is not a subdomain of the Target (see Section 3).
3. Merge all three into one list: `sort -u subfinder.txt amass.txt crtsh.txt`.
4. Find names unique to each tool with `comm -23 tool.txt <(sort -u other1.txt other2.txt)`.
5. Mark which tool found each name and whether it is in the program's scope (Appendix).

3. Reconciliation results

Almost every name was found by two tools at once; only four names came from a single tool.

Found by	Names	Share
Subfinder + crt.sh (not Amass)	212	92.6%
All three tools	12	5.2%
Subfinder only	4	1.7%
Amass + crt.sh (the apex domain `target.example`)	1	0.4%
crt.sh only (real names)	0	0%
Amass only	0	0%
Total unique names	**229**	100%

Names found by only one tool

Name	Found only by	Likely reason (Section 4)
`connect.target.example`	Subfinder	Never on its own certificate; known from passive DNS
`demo.next.target.example`	Subfinder	Covered by a wildcard certificate `*.next.target.example`
`maven.target.example`	Subfinder	Package repository, probably hosted by a third-party service with its own certificate
`pkgs.target.example`	Subfinder	Same as above

Removed as noise (10 crt.sh lines that are not subdomains of the Target)

Lines	What they really are
Four look-alike domains such as living--[target].com and future--[target].com, plus three www. versions (7)	Unrelated domains owned by others, whose names merely end in the same letters
verifiable[target].com (1)	A separate domain sharing a certificate with the Target; not a subdomain and out of scope
[target] document signing ecc ca 1 and ca 2 (2)	Certificate-authority names taken from certificate subject fields, not hostnames

The reasons in the second table are the most likely explanations, not confirmed. Running `subfinder -d target.example -cs` would show exactly which data source supplied each of the four names.

4. Why each tool found what the others missed

The results differ because each tool reads different data sources, and Subfinder already includes crt.sh among its sources.

Subfinder: largest list, and the only tool with unique names

Subfinder queries dozens of passive sources in parallel: Certificate Transparency (including crt.sh itself), passive DNS databases, search engines, web archives and threat-intelligence feeds. That explains both results:

- **It found almost everything crt.sh found**, because crt.sh is one of its sources.
- **It found 4 names crt.sh never saw.** A name only appears in Certificate Transparency if it was written on a public certificate. These four probably never were:
  - `demo.next.target.example` is most likely served by a wildcard certificate (`*.next.target.example`), which hides individual names.
  - `maven` and `pkgs` look like package repositories. These are often hosted by a third-party service that serves its own certificate, or are only resolved inside DNS records.
  - `connect` probably points (CNAME) to a third-party platform. Passive DNS datasets record the lookup even though no certificate names it.

crt.sh: high coverage, no unique real names, and noisy

Certificate Transparency is a public, append-only log of every TLS certificate issued. It is excellent for finding test and internal hosts, because companies often request a certificate for every environment. Here it found 225 real names, including many `dr` (disaster recovery), `preview` and `lowers` hosts. It found **nothing unique**, because Subfinder had already pulled the same data.

It was also the only noisy tool. The `%.` search pattern is a loose text match, so it returned **unrelated domains** that end in the same letters. Certificates also carry **non-hostname text** in their subject fields, such as certificate-authority names. Raw crt.sh output must therefore always be cleaned and checked.

Amass (passive): only 13 names, nothing new

Amass is powerful in active mode, where it brute-forces names, scrapes pages and builds a graph of related assets. In passive mode with **no API keys configured**, it can only reach a few free sources, and it stalled at 99.97% and was stopped early. So it found just 13 well-known names, all public-facing services such as the main website, support, demo and guides. All of them were found by the other tools too.

The only name Amass reported that Subfinder did not was the apex domain itself (`target.example`), because Subfinder lists subdomains only.

What this means in practice

Lesson	Evidence from this test
Tools that share sources overlap heavily	92.6% of names came from Subfinder and crt.sh together
A tool is only as good as its configuration	Amass without API keys found 5.7% of the total
No single source is complete	crt.sh missed 4 hosts not written on any certificate
Raw output needs cleaning	10 of 235 crt.sh lines (4.3%) were not subdomains
More tools mainly add confidence	Names confirmed by two independent sources are more likely real

5. Attack surface, scope and limitations

The 229 names reveal about ten separate environments; only 5 names, all in the sandbox, are in the program's scope.

Names by environment (second-level label)

Environment	Names	What it suggests
Directly under the domain	52	Public services: website, app, support, status, guides, VPN, certificate services
`next`	31	Pre-release / next version
`staging`	22	Staging
`sandbox` (in-scope environment)	21	Public test environment for developers
`it`	21	Corporate IT: VPN, single sign-on, firewalls, phishing-simulation and security tools
`preview` (incl. `dr.preview`)	20	Preview builds and their disaster-recovery copies
`next2`	14	A second pre-release track
`prod`	13	Production
`next2-dr`	12	Disaster recovery for `next2`
`lowers`, `prod-dr`, `ca`, others	23	Lower environments, production DR, certificate authority, misc.

Scope check. All 5 in-scope hosts were found: `api`, `app`, `keystone`, `monitoring` and `verify` under `sandbox.target.example`. The other 16 sandbox names, such as `admin`, `mcp`, `capture`, `notary` and `title`, are **out of scope** even though they share the environment. `title.sandbox` matches the unlisted host from Task 2, Finding 1.

Observations for the Target (informational, not tested)

- Public DNS names for corporate IT services (VPN, identity provider, firewalls, a phishing-simulation server) tell attackers which security products are in use. Consider moving them to private or split-horizon DNS.
- Many non-production environments are publicly named. Each one is extra attack surface that needs the same patching and access control as production.

Limitations

- Names were not resolved or probed, because most are out of scope. Some may be stale (no longer in DNS).
- Amass ran without API keys and was stopped early, so its result understates what it can do.
- Subfinder's exact version and its per-source breakdown (`-cs`) were not recorded, so the reasons for its four unique names are inferred rather than confirmed.

Ethics

Only passive third-party sources were used. No name in the list was visited, resolved against the Target's servers, scanned or brute-forced. Out-of-scope hosts appear in this report only as names.

Appendix: full deduplicated list

All 229 names, grouped by environment. Y = found by that tool; in scope = YES for the 5 hosts in the program's scope.

Hostname	Subfinder	Amass	crt.sh	In scope
target.example	-	Y	Y	no
admin.target.example	Y	-	Y	no
api.target.example	Y	Y	Y	no
keystone.api.target.example	Y	-	Y	no

Hostname	Subfinder	Amass	crt.sh	In scope
www.api.target.example	Y	-	Y	no
api-dev.target.example	Y	-	Y	no
api-nonprod.target.example	Y	-	Y	no
app.target.example	Y	Y	Y	no
auth.target.example	Y	-	Y	no
auth-dev.target.example	Y	-	Y	no
business.target.example	Y	Y	Y	no
lowers.ca.target.example	Y	-	Y	no
ocsp.lowers.ca.target.example	Y	-	Y	no
ocsp-lowers.ca.target.example	Y	-	Y	no
capture.target.example	Y	-	Y	no
cert.target.example	Y	-	Y	no
connect.target.example	Y	-	-	no
crl.target.example	Y	-	Y	no
demo.target.example	Y	Y	Y	no
dev.target.example	Y	-	Y	no
api.dev.target.example	Y	-	Y	no
auth.dev.target.example	Y	-	Y	no
email.target.example	Y	-	Y	no
sandbox.target.example	Y	-	Y	no
admin.sandbox.target.example	Y	-	Y	no
api.sandbox.target.example	Y	-	Y	YES
keystone.api.sandbox.target.example	Y	-	Y	no
api-ue2.sandbox.target.example	Y	-	Y	no
api-v2.sandbox.target.example	Y	-	Y	no
api-v2-ue2.sandbox.target.example	Y	-	Y	no
app.sandbox.target.example	Y	-	Y	YES
business.sandbox.target.example	Y	-	Y	no
capture.sandbox.target.example	Y	-	Y	no
csp.sandbox.target.example	Y	-	Y	no
go.sandbox.target.example	Y	-	Y	no
keystone.sandbox.target.example	Y	-	Y	YES
lender.sandbox.target.example	Y	-	Y	no
link.sandbox.target.example	Y	-	Y	no
mcp.sandbox.target.example	Y	-	Y	no

Hostname	Subfinder	Amass	crt.sh	In scope
monitoring.sandbox.target.example	Y	-	Y	YES
notary.sandbox.target.example	Y	-	Y	no
paravision.sandbox.target.example	Y	-	Y	no
paravision-ue2.sandbox.target.example	Y	-	Y	no
title.sandbox.target.example	Y	-	Y	no
verify.sandbox.target.example	Y	-	Y	YES
go.target.example	Y	-	Y	no
content.guides.target.example	Y	Y	Y	no
data.guides.target.example	Y	Y	Y	no
hello.target.example	Y	-	Y	no
inquiry.target.example	Y	Y	Y	no
it.target.example	Y	-	Y	no
1pass.it.target.example	Y	-	Y	no
admin-phish.it.target.example	Y	-	Y	no
browser.it.target.example	Y	-	Y	no
clops.it.target.example	Y	-	Y	no
clops2.it.target.example	Y	-	Y	no
defectdojo.it.target.example	Y	-	Y	no
epp.it.target.example	Y	-	Y	no
fw.it.target.example	Y	-	Y	no
www.fw.it.target.example	Y	-	Y	no
fw-2.it.target.example	Y	-	Y	no
www.fw-2.it.target.example	Y	-	Y	no
fw2.it.target.example	Y	-	Y	no
www.fw2.it.target.example	Y	-	Y	no
fw2-2.it.target.example	Y	-	Y	no
www.fw2-2.it.target.example	Y	-	Y	no
gophish.it.target.example	Y	-	Y	no
oidc.it.target.example	Y	-	Y	no
okta.it.target.example	Y	-	Y	no
oidc.prod.it.target.example	Y	-	Y	no
tac.it.target.example	Y	-	Y	no
vpn3.it.target.example	Y	-	Y	no
keystone.target.example	Y	-	Y	no
lender.target.example	Y	-	Y	no

Hostname	Subfinder	Amass	crt.sh	In scope
link.target.example	Y	Y	Y	no
lowers.target.example	Y	-	Y	no
cert.lowers.target.example	Y	-	Y	no
crl.lowers.target.example	Y	-	Y	no
notapki.lowers.target.example	Y	-	Y	no
notapki-ue2.lowers.target.example	Y	-	Y	no
ocsp.lowers.target.example	Y	-	Y	no
lp.target.example	Y	-	Y	no
maven.target.example	Y	-	-	no
mcp.target.example	Y	-	Y	no
metabase.target.example	Y	-	Y	no
next.target.example	Y	-	Y	no
admin.next.target.example	Y	-	Y	no
api.next.target.example	Y	-	Y	no
keystone.api.next.target.example	Y	-	Y	no
api-ue2.next.target.example	Y	-	Y	no
api-v2.next.target.example	Y	-	Y	no
api-v2-ue2.next.target.example	Y	-	Y	no
app.next.target.example	Y	-	Y	no
business.next.target.example	Y	-	Y	no
capture.next.target.example	Y	-	Y	no
classy-gpu.next.target.example	Y	-	Y	no
conclusive-light.next.target.example	Y	-	Y	no
csp.next.target.example	Y	-	Y	no
demo.next.target.example	Y	-	-	no
features.next.target.example	Y	-	Y	no
globotron.next.target.example	Y	-	Y	no
go.next.target.example	Y	-	Y	no
keystone.next.target.example	Y	-	Y	no
lender.next.target.example	Y	-	Y	no
link.next.target.example	Y	-	Y	no
mcp.next.target.example	Y	-	Y	no
metabase.next.target.example	Y	-	Y	no
monitoring.next.target.example	Y	-	Y	no
notapki.next.target.example	Y	-	Y	no

Hostname	Subfinder	Amass	crt.sh	In scope
notary.next.target.example	Y	-	Y	no
paravision.next.target.example	Y	-	Y	no
paravision-ue2.next.target.example	Y	-	Y	no
relay.next.target.example	Y	-	Y	no
relay-ue2.next.target.example	Y	-	Y	no
router.next.target.example	Y	-	Y	no
title.next.target.example	Y	-	Y	no
verify.next.target.example	Y	-	Y	no
next2.target.example	Y	-	Y	no
admin.next2.target.example	Y	-	Y	no
api.next2.target.example	Y	-	Y	no
keystone.api.next2.target.example	Y	-	Y	no
api-v2.next2.target.example	Y	-	Y	no
app.next2.target.example	Y	-	Y	no
business.next2.target.example	Y	-	Y	no
capture.next2.target.example	Y	-	Y	no
go.next2.target.example	Y	-	Y	no
keystone.next2.target.example	Y	-	Y	no
lender.next2.target.example	Y	-	Y	no
link.next2.target.example	Y	-	Y	no
notary.next2.target.example	Y	-	Y	no
title.next2.target.example	Y	-	Y	no
verify.next2.target.example	Y	-	Y	no
admin.next2-dr.target.example	Y	-	Y	no
api.next2-dr.target.example	Y	-	Y	no
app.next2-dr.target.example	Y	-	Y	no
business.next2-dr.target.example	Y	-	Y	no
capture.next2-dr.target.example	Y	-	Y	no
go.next2-dr.target.example	Y	-	Y	no
keystone.next2-dr.target.example	Y	-	Y	no
lender.next2-dr.target.example	Y	-	Y	no
link.next2-dr.target.example	Y	-	Y	no
notary.next2-dr.target.example	Y	-	Y	no
title.next2-dr.target.example	Y	-	Y	no
verify.next2-dr.target.example	Y	-	Y	no

Hostname	Subfinder	Amass	crt.sh	In scope
notary.target.example	Y	-	Y	no
ocsp.target.example	Y	-	Y	no
oncall.target.example	Y	-	Y	no
pkgs.target.example	Y	-	-	no
admin.preview.target.example	Y	-	Y	no
api.preview.target.example	Y	-	Y	no
app.preview.target.example	Y	-	Y	no
business.preview.target.example	Y	-	Y	no
capture.preview.target.example	Y	-	Y	no
admin.dr.preview.target.example	Y	-	Y	no
app.dr.preview.target.example	Y	-	Y	no
business.dr.preview.target.example	Y	-	Y	no
capture.dr.preview.target.example	Y	-	Y	no
keystone.dr.preview.target.example	Y	-	Y	no
lender.dr.preview.target.example	Y	-	Y	no
notary.dr.preview.target.example	Y	-	Y	no
title.dr.preview.target.example	Y	-	Y	no
verify.dr.preview.target.example	Y	-	Y	no
keystone.preview.target.example	Y	-	Y	no
lender.preview.target.example	Y	-	Y	no
link.preview.target.example	Y	-	Y	no
notary.preview.target.example	Y	-	Y	no
title.preview.target.example	Y	-	Y	no
verify.preview.target.example	Y	-	Y	no
api.prod.target.example	Y	-	Y	no
keystone.api.prod.target.example	Y	-	Y	no
api-ue2.prod.target.example	Y	-	Y	no
api-v2.prod.target.example	Y	-	Y	no
api-v2-ue2.prod.target.example	Y	-	Y	no
csp.prod.target.example	Y	-	Y	no
go.prod.target.example	Y	-	Y	no
link.prod.target.example	Y	-	Y	no
monitoring.prod.target.example	Y	-	Y	no
notapki.prod.target.example	Y	-	Y	no
notapki-ue2.prod.target.example	Y	-	Y	no

Hostname	Subfinder	Amass	crt.sh	In scope
paravision.prod.target.example	Y	-	Y	no
paravision-ue2.prod.target.example	Y	-	Y	no
api.prod-dr.target.example	Y	-	Y	no
api-v2.prod-dr.target.example	Y	-	Y	no
go.prod-dr.target.example	Y	-	Y	no
link.prod-dr.target.example	Y	-	Y	no
real-ca.target.example	Y	-	Y	no
security.target.example	Y	-	Y	no
staging.target.example	Y	-	Y	no
admin.staging.target.example	Y	-	Y	no
api.staging.target.example	Y	-	Y	no
keystone.api.staging.target.example	Y	-	Y	no
api-ue2.staging.target.example	Y	-	Y	no
api-v2.staging.target.example	Y	-	Y	no
api-v2-ue2.staging.target.example	Y	-	Y	no
app.staging.target.example	Y	-	Y	no
business.staging.target.example	Y	-	Y	no
capture.staging.target.example	Y	-	Y	no
csp.staging.target.example	Y	-	Y	no
demo.staging.target.example	Y	-	Y	no
go.staging.target.example	Y	-	Y	no
keystone.staging.target.example	Y	-	Y	no
lender.staging.target.example	Y	-	Y	no
link.staging.target.example	Y	-	Y	no
mcp.staging.target.example	Y	-	Y	no
monitoring.staging.target.example	Y	-	Y	no
notary.staging.target.example	Y	-	Y	no
paravision.staging.target.example	Y	-	Y	no
paravision-ue2.staging.target.example	Y	-	Y	no
title.staging.target.example	Y	-	Y	no
verify.staging.target.example	Y	-	Y	no
stampede.target.example	Y	-	Y	no
static.target.example	Y	-	Y	no
status.target.example	Y	-	Y	no
support.target.example	Y	Y	Y	no

Hostname	Subfinder	Amass	crt.sh	In scope
title.target.example	Y	Y	Y	no
try.target.example	Y	-	Y	no
verification.target.example	Y	-	Y	no
verify.target.example	Y	Y	Y	no
vpn.target.example	Y	-	Y	no
www.vpn.target.example	Y	-	Y	no
vpn2.target.example	Y	-	Y	no
www.vpn2.target.example	Y	-	Y	no
vpn3.target.example	Y	-	Y	no
www.vpn3.target.example	Y	-	Y	no
www.target.example	Y	Y	Y	no
www-staging.target.example	Y	-	Y	no
x401.target.example	Y	-	Y	no
mcp-sandbox.x401.target.example	Y	-	Y	no
x401-test.target.example	Y	-	Y	no

Task 4 — Scanner Comparison: OWASP ZAP vs Burp Suite Professional

1. Summary

Both scanners found every major vulnerability class in Mutillidae: SQL injection, reflected XSS, OS command injection, path traversal and file inclusion. Burp went deeper into each page, testing headers, cookies and JavaScript. ZAP went wider, covering more of the server and reporting more configuration problems.

	OWASP ZAP	Burp Suite Professional
Version	2.17.0	v2026.4.3 (build 47818)
Scan time	75 min (10:20–11:35)	65 min (12:35–13:40)
Issue types reported	30	30
By severity (ZAP: types; Burp: instances)	High 7, Medium 7, Low 9, Info 7	High 36, Medium 2, Low 15, Info 66
Pages outside `/mutillidae/` reached	Yes: `/dav`, `/phpMyAdmin`, `/dvwa`, `/twiki`	No
Tested HTTP headers and cookies	No (default settings)	Yes
Out-of-band detection	No	Yes (Burp Collaborator)

The raw totals cannot be compared directly. ZAP counts every URL where an alert fired, while Burp groups results by path and input point, so the comparison below matches findings by **vulnerability, page and parameter**.

2. Lab setup and method

Both scanners ran from the same Windows 11 host against the same deliberately vulnerable app, with no login and default scan settings.

Item	Detail
Target	OWASP Mutillidae (the version bundled with Metasploitable 2), `http://192.168.40.130/mutillidae/`
Network	VMware NAT. The VM is not reachable from the LAN. It could reach the internet, which Burp's out-of-band checks relied on.
Scanner host	Windows 11 (same machine for both tools)
Authentication	None. Both scans were unauthenticated.
ZAP mode	Quick Start Automated Scan: traditional spider, then active scan with the default policy
Burp mode	Dashboard → New scan → Crawl and audit, default configuration, 5 concurrent requests
Order	ZAP first (11 Nov, 10:20), Burp second (12:35)
Database reset	Mutillidae's database was reset before the Burp scan, so both tools started from the same state

Fairness controls and known differences

- Same target URL, same host machine, same network path and no credentials for either tool.
- Similar time budgets: 75 minutes for ZAP, 65 minutes for Burp.

- Default settings were kept on purpose, to compare the tools as most users would run them. Several differences below come from those defaults, not from what each tool can do when configured.
- ZAP's first attempt froze a Kali VM after 2 hours. Moving the scanners to the Windows host fixed it.
- Both scanners requested `set-up-database.php`, the page that resets Mutillidae's database, because no exclusions were set. Data created during a scan may therefore have been wiped partway through.

3. Found by both scanners

The serious vulnerabilities overlap almost completely; the scanners differ mainly in naming and severity.

Vulnerability	Where (page → parameter)	ZAP name and risk	Burp name and severity
OS command injection	`dns-lookup.php` → `target_host`	Remote OS Command Injection — High	OS command injection — High, Certain
SQL injection	`user-info.php` → `username`, `password`; `view-someones-blog.php` → `author`; `add-to-your-blog.php` → `blog_entry`	SQL Injection - MySQL — High	SQL injection — High, Certain
Path traversal / local file inclusion	`index.php` → `page`; `source-viewer.php` → `page`, `phpfile`	Path Traversal — High	File path traversal (High, Firm) and File path manipulation (High, Certain)
Remote file inclusion / server fetches external URLs	`text-file-viewer.php` → `textfile`	Remote File Inclusion — High	Out-of-band resource load (HTTP) and External service interaction (HTTP, DNS) — High
Reflected XSS	`page`, `choice`, `forwardurl`, `background_color`, `target_host`, `phpfile`, `textfile`, `username` on their pages	Cross Site Scripting (Reflected) — High	Cross-site scripting (reflected) — High, Certain
Open redirect	`redirectandlog.php` → `forwardurl`	External Redirect — **High**	Open redirection (reflected) — **Low**
Missing CSRF protection	Blog and login forms	Absence of Anti-CSRF Tokens — **Medium**	Cross-site request forgery — **Information**
Clickjacking	All pages	Missing Anti-clickjacking Header — **Medium**	Frameable response — **Information**
Vulnerable jQuery	`javascript/ddsmoothmenu/jquery.min.js`	Vulnerable JS Library — Medium	Vulnerable JavaScript dependency — Low, Tentative

Vulnerability	Where (page → parameter)	ZAP name and risk	Burp name and severity
Session cookie without HttpOnly	`PHPSESSID`	Cookie No HttpOnly Flag — Low	Cookie without HttpOnly flag set — Low
Private IP disclosure	Several pages	Private IP Disclosure — Low	Private IP addresses disclosed — Information

Severity disagreements. The tools rate the same flaw differently. ZAP rates the open redirect High, while Burp rates it Low, because a redirect alone does not compromise the server. ZAP also rates missing CSRF tokens and clickjacking as Medium, while Burp treats them as Information until someone shows real impact. Scanner severity is a starting point; a tester must judge real risk in context.

4. Found only by ZAP, and why Burp missed it

ZAP's extra findings come from a wider crawl and from checks Burp deliberately leaves out.

Finding (ZAP risk)	Where	Why Burp did not report it
SQL injection on `login.php` and `register.php` (High)	`username`, `password`, `my_signature`	**Probably a reporting difference, not a miss.** Every Mutillidae page is `index.php?page=…`, and Burp groups issues by path and parameter name. So `login.php`'s `username` and `user-info.php`'s `username` may have been merged into one Burp issue. This would need Burp's per-request evidence to confirm.
Directory browsing (Medium), error disclosure (Medium)	`/dav/`	**Crawl scope.** ZAP's spider followed links out of `/mutillidae/` to the WebDAV folder, phpMyAdmin, DVWA and TWiki. Burp stayed inside the start folder.
PHP version with known vulnerabilities (High)	Server header (PHP 5.2.4)	**Different check sets.** ZAP flags outdated software versions from headers. Burp's default scan only reports outdated versions in JavaScript libraries.
Application error disclosure (Medium), debug error messages (Low)	Blog, log and user-info pages	ZAP has passive rules for PHP and SQL error text in pages. In Burp, the same errors show up as evidence inside the SQL injection findings rather than as separate issues.
Missing CSP header (Medium), missing X-Content-Type-Options (Low), cookie without SameSite (Low)	All pages	**Different philosophy.** ZAP reports missing security headers as alerts. Burp mostly does not, to reduce noise.

Finding (ZAP risk)	Where	Why Burp did not report it
Server and X-Powered-By version leaks, in-page banner (Low)	All pages	Same reason: ZAP reports information disclosure that Burp treats as background detail.
XSLT Injection (Medium, 15)	`index.php → page`; `text-file-viewer.php → textfile`	**Likely a false positive.** These are the same parameters as the file-inclusion bugs. ZAP's XSLT payload made the server fetch a URL through PHP's `include`, and ZAP read that as XSLT processing. Burp correctly classified the same behaviour as path traversal and out-of-band resource load.
Suspicious comments (21), timestamp disclosure, "modern web application", authentication and session detection (Info)	Various	ZAP's informational rules that describe the app; Burp has no direct equivalents.

5. Found only by Burp, and why ZAP missed it

Burp's extra findings come from testing more input points, from out-of-band detection and from analysing JavaScript.

Finding (Burp severity)	Where	Why ZAP did not report it
SQL injection through the **Referer** and **User-Agent** headers (High, Certain)	`capture-data.php`	**Input points.** Mutillidae writes these headers into its database. Burp injects into headers by default; ZAP's default active scan does not test HTTP headers (Options → Active Scan Input Vectors).
SQL injection and XSS through the **name** of an extra URL parameter (High)	`capture-data.php`	Burp adds its own parameter names and tests them. ZAP only tests parameters that already exist.
Reflected XSS in more inputs: `author`, `blog_entry`, `csrf-token`, `password`, `username`, the `showhints` cookie, and the Referer and User-Agent headers	Blog, login and user-info pages	Same reason: Burp tests cookies and headers by default, and its XSS checks confirm reflection in more contexts.
Path traversal in `textfile` (High)	`text-file-viewer.php`	ZAP found this parameter but reported it as Remote File Inclusion and XSLT Injection instead. It is the same flaw under a different name, not a true miss.
Out-of-band DNS and HTTP interaction, and command injection confirmed by DNS lookup	`dns-lookup.php`, `text-file-viewer.php`	**Burp Collaborator.** Burp made the server contact a Burp-controlled domain (`oastify.com`), which proves blind vulnerabilities. ZAP can do this only with its OAST add-on configured, which it was not.

Finding (Burp severity)	Where	Why ZAP did not report it
DOM-based XSS (Medium, Tentative), DOM-based open redirect and client-side JSON injection (Low)	`index.php` JavaScript	**JavaScript analysis.** Burp traces data from sources such as `location` and `postMessage` to dangerous sinks in the browser. ZAP's DOM XSS rule is an optional add-on that needs a browser-based crawl.
Cleartext submission of password (High), password sent by GET (Low), unencrypted communications (Low)	Login and user-info forms	**Transport checks.** Burp flags credentials sent over plain HTTP. ZAP's default rules do not report a site that runs only over HTTP.
HTTP TRACE method enabled (Information)	Server	Burp sends a TRACE request; ZAP's equivalent check is not in the default policy.
Email addresses disclosed, cross-domain Referer leakage, path-relative style sheet import, client-side parameter pollution, suspicious input transformation (Low/Information)	Various pages	Smaller Burp-specific checks with no direct equivalent in ZAP's default policy.

6. Why they differ, and conclusion

Almost every difference traces back to five causes: crawl scope, which inputs each tool tests, out-of-band detection, JavaScript analysis and reporting philosophy.

Cause	Effect in this test
Crawl scope	ZAP's spider left `/mutillidae/` and found WebDAV directory listings and phpMyAdmin pages. Burp stayed within the start folder.
Input points tested	Burp injected into headers, cookies and parameter names and found SQL injection in the Referer and User-Agent headers. ZAP tested only query and form parameters.
Out-of-band detection	Burp Collaborator confirmed server-side DNS and HTTP requests. ZAP had no OAST server configured.
JavaScript analysis	Burp reported DOM-based XSS, open redirect and JSON injection. ZAP reported none.
Reporting philosophy	ZAP reports missing headers, version leaks and errors as separate alerts. Burp keeps those quiet, but flags cleartext passwords. Each tool also assigns severity differently.
Grouping and naming	ZAP lists each URL; Burp groups by path and input point. The same flaw can appear under different names, such as RFI in ZAP versus out-of-band resource load in Burp.

False positives. ZAP's 15 "XSLT Injection" alerts are most likely the file-inclusion flaw under the wrong name. Burp's DOM-based issues are marked Tentative and need manual checking in a browser. Neither tool's output should be reported to a client without manual confirmation.

What neither scanner found. Both scans were unauthenticated, so pages behind login were tested only partly. Neither reported **stored XSS** in the blog, even though entries are saved and shown to other users. Neither tested **access control or business logic**, such as whether one user can view another user's data, because scanners cannot understand what the application is supposed to allow.

Conclusion. On this target, Burp was more thorough per page: it tested more input points, confirmed findings out-of-band and analysed client-side code. ZAP covered more of the server and gave more configuration feedback. The tools complement each other. A practical approach is to run ZAP for

broad, free, repeatable coverage (for example in a CI pipeline), use Burp for deep testing of important pages, and then confirm and extend both manually.

Improvements for a repeat test

- Run both scanners authenticated, with the same user account.
- Exclude `set-up-database.php` and logout links in both tools.
- In ZAP, enable header and cookie input vectors, the OAST add-on and the DOM XSS rule, so the comparison reflects capability rather than defaults.
- Limit both crawls to `/mutillidae/` so the scope is identical.